

Can a clinic afford to run its own coding model?

Dr. Findus, our startup in Berlin

German GPs code every consultation.

We propose, the physician confirms.

Reliability is the product.

Today it runs on hosted APIs.

Could it run on our own hardware?

Qwen3 + LoRA on CodiEsp.

1 000 clinical case reports, ICD-10, CC-BY 4.0.

The resource problem

8 GB of weights at 4B before a single activation. Cases up to 2 489 tokens.

One JAX program, lowered by XLA to CPU, GPU and TPU.

Where does the time go, and what does it take to run this at all?

One program. Three backends. All measured.

Compilation

One JAX program
XLA lowers it to all three
LoRA via qwix, Tunix trainer

Orchestration

3 pinned Docker images
Kubernetes across 2 clusters
Kueue admission, explicit limits

Storage

gcsfuse CSI, implicit-dirs
8 GB checkpoint from a bucket
File cache left as a knob

Backend	Hardware	Host arch	How the code got there
CPU	32-core x86_64, 31 GiB	x86_64	bare node
GPU	NVIDIA GH200 480 GB	ARM64 Grace	public image + ConfigMap
TPU	v5e 2x4, 8 chips	x86_64	private registry + gcsfuse

The GH200 host is ARM64. That one fact cost five separate blockers.

We did not time the job. We took it apart.

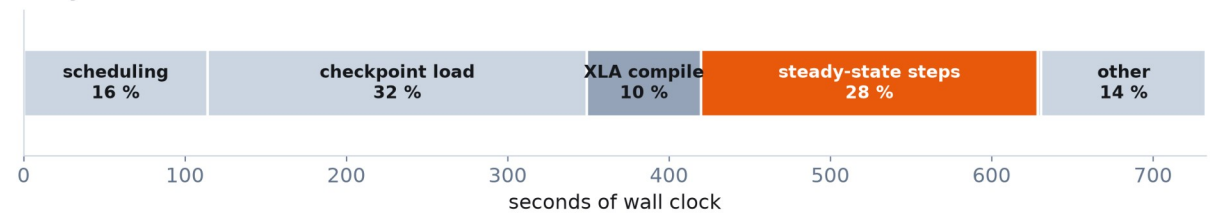
Six phases per run, not one total

Sweeps walked into OOM, failed cells kept

One schema, one JSON per run

Every figure reads only those files

Only 28 % of 733 s is arithmetic



The schema caught a lie

93 μ s per step. 11 M tokens/s. Impossible, yet it carried status "ok".

Its own notes admitted it may have timed dispatch, not completion. Discarded.

RESULTS

4

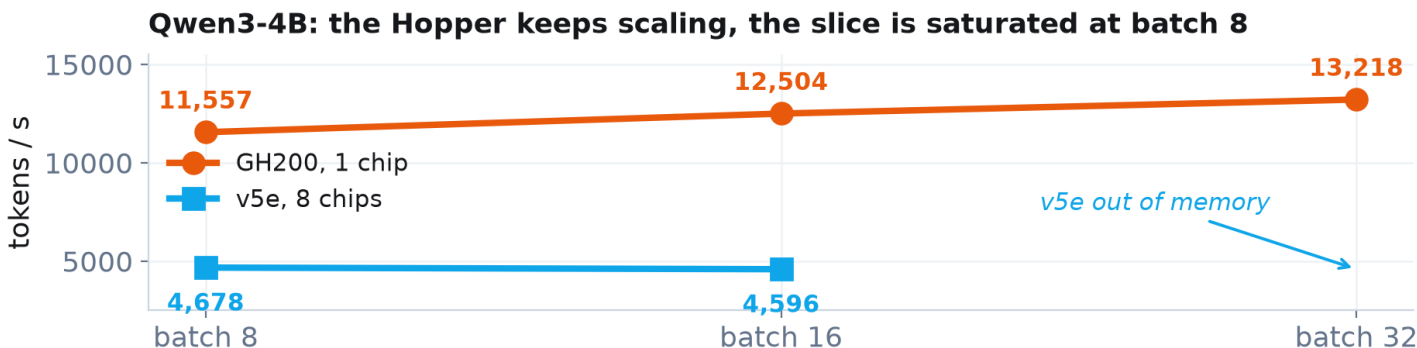
Under load, one GH200 beats eight TPU chips.

Qwen3-4B, seq 1024	GH200 1 chip	v5e 8 chips	ratio
batch 8	11,557 tok/s	4,678 tok/s	2.5x
batch 16	12,504 tok/s	4,596 tok/s	2.7x
batch 32	13,218 tok/s	out of memory	
batch 64	out of memory		

2.7x

the throughput of an eight-chip v5e slice, from a single Hopper chip

At batch 1 both chips idle and tie to within 0.23 %. That tie was an artefact.



Memory boundary

GH200 fails between 32 and 64
v5e fails between 16 and 32

The mitigation, controlled

File cache: load 14.6x faster,
31 % off wall clock.
Step time unchanged to the
fourth decimal.

The chip was never the bottleneck.

28 %

of wall clock is arithmetic

Of 733 s, only 209 s computes.
The biggest single phase is
reading the checkpoint.

6.7×

more parameters, CPU stopped

0.6B to 4B did not make the CPU
6.7× slower. XLA never finished
compiling; the kernel killed it
at 31.5 GB.

1 of 36

dependencies were TPU-bound

The GH200 sat idle behind an
ARM64 host, a QEMU segfault
and a registry 403. Swapping
one pin is the whole port.

Scaling recommendation

Do not scale out. Amortise: raise batch only until
the chip is occupied, then attack fixed cost:
persistent compile caches, warm workers instead of
a pod per job, file cache on anything reading a
checkpoint.

What this means for Dr. Findus

On the real 4B workload one GH200 runs 2.7x faster
than eight TPU chips and costs 10x less:
\$0.56 against \$5.72 per 1 000 steps.

Self-hosting is not the cost problem we assumed.
The real cost is fixed overhead per job.